

MiniKV

Introducción

Los [key-value stores \(kvs\)](#) son una de las estructuras de persistencia más utilizadas en sistemas modernos, debido a su simplicidad, eficiencia y escalabilidad.

En este ejercicio se propone implementar un **mini kvs persistente** en Rust llamado `minikv`. El sistema permitirá almacenar pares **clave-valor** utilizando un **archivo de log** donde se registran todas las operaciones, además de soportar la creación de **snapshots** que permitan compactar el log.

Interfaz

El programa deberá ejecutarse mediante la línea de comandos, y soportará los siguientes comandos:

- `set`
- `get`
- `length`
- `snapshot`

Comando `set`

Asocia un valor a una clave.

```
minikv set <clave> <valor>
```

Por ejemplo:

```
$ minikv set clave1 valor1
OK

$ minikv set clave2 valor2
OK
```

Si se omite el valor, entonces se desasocia el valor de la clave (unset).

```
$ minikv set clave2  
OK
```

Comando **get**

Obtiene el valor asociado a una clave.

```
minikv get <clave>
```

Por ejemplo:

```
$ minikv get clave1  
valor1  
  
$ minikv get clave2  
NOT FOUND
```

Comando **length**

Devuelve la cantidad de claves con valor.

```
minikv length
```

Por ejemplo:

```
$ minikv length  
1  
  
$ minikv set clave1  
OK  
  
$ minikv length  
0
```

Comando **snapshot**

Genera un snapshot del estado actual y compacta el log.

```
minikv snapshot
```

Persistencia

El sistema utilizará dos archivos del directorio actual para persistir información:

- `.minikv.log`
- `.minikv.data`

El log contiene todas las operaciones de escritura desde el último snapshot. Por ejemplo:

```
set clave1 valor1
set clave2 valor2
set clave2
set clave1
```

Al ser un *append-only log*, solo se tiene permitido escribir al final del log, sin modificar las operaciones previas.

El snapshot contiene el estado completo del store. Por ejemplo:

```
clave1 valor1
clave2 valor2
```

Para generar un snapshot, se deberá:

1. Leer `.minikv.data` si existe y cargar los datos en memoria.
2. Leer `.minikv.log` y aplicar secuencialmente todas las operaciones.
3. Persistir el estado completo nuevamente en `.minikv.data`.
4. Truncar el log `.minikv.log`.

Es muy importante que estos archivos respeten el formato indicado.

Restricciones

- No se permite que el programa lance un `panic!()`. Es decir, no se puede utilizar `.unwrap()` o `.expect()`. Todo caso de error deberá manejarse idiomáticamente con las estructuras y funciones brindadas por el lenguaje.
- No se permite utilizar la función `exit()`. Se deberá salir del programa finalizando el scope de la función `main`.
- No se permite utilizar el módulo `mem` para la manipulación de memoria.
- Para realizar un uso adecuado de memoria y respetar las reglas de *ownership* se deberá evitar el uso de `.clone()` y `.copy()` en las estructuras principales de datos.

Requerimientos no funcionales

Los siguientes son los requerimientos no funcionales para la resolución del proyecto:

- El proyecto deberá ser desarrollado en la última versión estable de Rust (1.94), usando las herramientas de la biblioteca estándar.
- Se deben implementar tests unitarios.
- No se permite utilizar *crates* externos.
- El código fuente debe compilarse en la versión estable del compilador.
- No se permite utilizar bloques `unsafe`.
- El código deberá funcionar en ambiente Unix / Linux.
- Los programas deberán ejecutarse en la línea de comandos.
- La compilación no debe arrojar `warnings` del compilador, ni del *linter* `clippy`.
- Las funciones y los tipos de datos (`struct` , `enum`) deben estar documentados siguiendo el estándar de `cargo doc`.
- El código debe formatearse utilizando `cargo fmt`.
- Las funciones no deben tener una extensión mayor a 30 líneas. Si se requiriera una extensión mayor, se deberá particionarla en varias funciones.
- Cada tipo de dato implementado debe ser colocado en un módulo (archivo) independiente.

El proyecto deberá realizarse de manera individual. Cualquier tipo de copia significa la expulsión automática de la materia. No está permitido el uso de código generado por ninguna IA, ni copiar código de soluciones existentes en Internet.

Entrega

Fecha de entrega: 23/3 - 18:00

La entrega se realizará por Algotron. Para que la entrega se considere válida, deberán pasar todas las verificaciones de la plataforma. En caso contrario, no se podrá continuar en la materia.